Facebook Posts

Justin Mundell

12/6/2022

Introduction

Facebook is used by millions of people daily and has a diverse user base globally. The data set being used for this project serves as a collection of information on customer engagement in the different forms of posts for businesses to utilize on social media. It consists of over 7,000 Facebook posts that are labeled as videos, photos, links, and status updates. All posts were taken from the Facebook pages of 10 Thai fashion and cosmetics retail sellers from March 2012 to 2018. Additionally, it was directly sourced from the Facebook API. Metrics of individuals' engagement with Facebook posts were recorded consisting of shares, comments, and emoji reactions. The emoji reactions include reactions such as "love", "wow", "haha", "sad", and "angry". These reactions are meant to be distinguished from traditional "likes" in social media.

Although this data is limited to posts from Thailand, users' interactions with the posts can be from practically anyone. After the introduction of live streaming around April 2016, Facebook users have been able to interact with posts in a variety of ways because of emojis. Emojis are graphics made up of text characters used to represent an emotion. It is important to note that the number of emojis in the data set is going to be low since they were not used prior to the introduction of Facebook Live in early 2016 and only account for 2 years of the data. This data set will provide us with insight into how consumers engage with different types of posts.

For the focus of the project, finding potential groupings and separability in the data while seeing what predictors are highly correlated will be prioritized. The main goal is to determine the optimal number of groups and to see if they correspond to the different types of labels for Facebook posts. My data question is: Will methods such as K-means and hierarchical clustering create clusters for our expected labels and how much variation can be explained by the data?

Description of variables:

- 'status_id': The identification number for each post
- 'status_type': Types of posts such as links, videos, photos, and status updates
- 'status_published': The specific date a post was published
- 'num_reactions': The number of times a post was reacted to
- 'num_comments': The number of comments a post has
- 'num_shares': The number of times a post was shared
- 'num_likes': The traditional number of likes a post will have
- 'num_loves': The number of love emojis per post
- 'num_wows': The number of wow emojis per post
- 'num_hahas': The number of haha emojis per post
- 'num_sads': The number of sad emojis per post
- 'num_angrys': The number of angry emojis per post.

Methodology

The principal component analysis is useful for data visualization since it produces a low-dimensional representation of the data while also providing interesting data patterns. A sequence of linear combinations of the explanatory variables will be configured into principal components. The first principal component has maximum variance among all linear combinations while the remaining components account for as much of the remaining variation as possible and are all uncorrelated with each other. An advantage of this is how the principal component loading vectors define a direction in the feature space that demonstrates the proportion of variance explained by each subsequent principal component. One disadvantage is that there isn't an optimal way to decide how many principal components are enough to use.

K-means clustering is a simple approach for partitioning a data set into 'K' distinct, non-overlapping clusters. After specifying the number of K clusters, the algorithm can assign each observation to an initial cluster assignment. Iterative updates are then provided to the cluster assignments until they stop changing. These updates are performed by computing the cluster centroid and assigning each observation to the cluster whose centroid is closest through Euclidean distance. On the surface, the simplicity of the mathematical process makes it seem advantageous. However, a disadvantage is the pre-specification of the number of clusters K since selecting the number of clusters beforehand can result in the wrong number of clusters forming.

Moreso, the within-cluster variation for the K-means algorithm measures how much observations within a cluster differ from each other. For observations to be considered well-clustered, the within-cluster variation must be as small as possible. It's defined through the squared Euclidean distance between all pairs of observations, giving us the main optimization task of clustering:

$$\underset{C_1,\ldots,C_k}{minimize} \left\{ \sum_{k=1}^{K} \frac{1}{|C_k|} \sum_{i,j \in C_k,\ i>j} \sum_{r=1}^{p} (x_{ir} - x_{jr})^2 \right\}$$

An alternative approach is hierarchical clustering. It does not require us to know the number of clusters in advance resulting in no random initializations and acquiring the same clustering solution. It also results in a tree-based representation of the observations called a Dendrogram. The process of hierarchical clustering begins by treating each observation as its own cluster and eventually merging the most similar clusters into one cluster based on their dissimilarities that are measured by Euclidean distance.

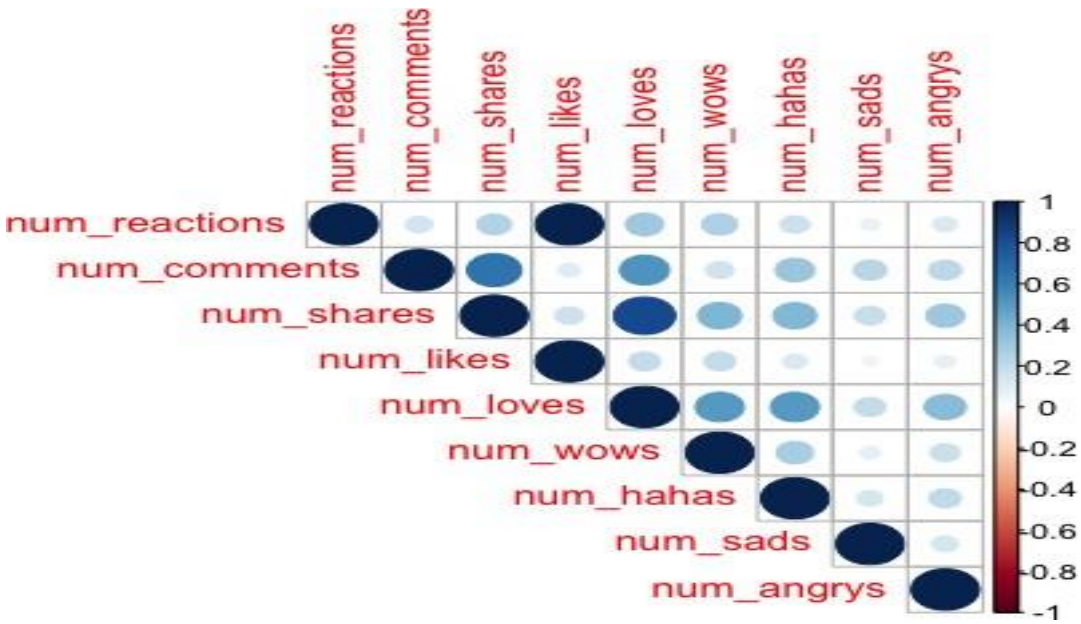
Choices of linkages can help measure the dissimilarity between two clusters of observations. Three linkage methods that will be used are complete, single, and average linkage. Complete linkage measures the maximal inter-cluster dissimilarity while single linkage measures the minimal inter-cluster dissimilarity. Unlike the other two, average linkage measures the mean inter-cluster dissimilarity. A disadvantage of this method is that it is computationally slow. This is because it builds a long hierarchy of clustering solutions and must calculate many dissimilarity matrices. Another disadvantage is how the results might heavily depend on the choice of linkage and that the Dendrogram may not be representable depending on the number of observations in a data set.

Analysis

Exploratory Data Analysis (EDA)

There were four columns that provided no information or values, so they were removed. The variable 'status_published' was also removed since this variable would require time series analysis. Another variable, 'status_id', was also removed since the analysis won't require knowing the specific identification number of each post. Following the removal of these variables, the data set contains 7,050 observations and 10 total predictors.

From the correlation matrix, some of the variables are highly correlated (with a correlation value over 0.8). Some of the predictors being highly correlated could lead to a multicollinearity problem for the creation of any future models. The variables 'num_shares' and 'num_reactions' are highly correlated with a value of 0.995. Also, variables 'num_loves' and 'num_shares' are highly correlated with a value of 0.820. Below is a visual representation of the correlation matrix. The darker the blue the higher the correlation is between two numeric variables corresponding with the correlation matrix as shown in the EDA appendix.



Scaling the data is going to be necessary for principal component analysis and both clustering methods. Extreme values exist in the data set which can be seen within the summary of the data as shown in the EDA section of the appendix. For instance, the maximum number of comments on a Facebook post was 20,990 while the maximum number of angry emojis was 31. Not scaling the data before performing principal component analysis would result in most of the principal components being driven by the different types of interaction predictors (num_comments…) instead of the emojis. This is because the interaction predictors have the largest mean and variance. Clustering results could also widely vary and be misleading without scaling.

Separability between photos and videos exists within the data. A plot for 'num_reactions' and 'num_comments' show the separation between both types of posts. Another

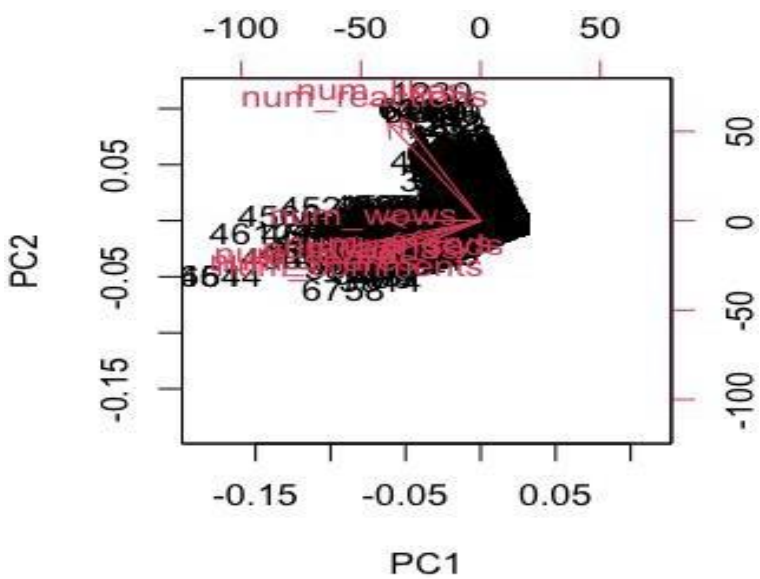
plot for ‘num_shares’ and ‘num_likes’ are similar in separability as well. There is no visible separability between the link and status which corresponds to the data summary. Only 63 observations for links and 365 for status occur in the data. Noting that videos and photos are going to be the most popular forms of posts on Facebook is important going forward since there are possibly only two groups forming from the data. View the EDA appendix to view the separability between the predictors.

Principal Component Analysis

From the summary of the *pc.out()* function shown in the PCA section of the appendix, we can expect a min(n-1, p)= 9 different principal components. After obtaining 9 principal components, the first principal component explains about 38.33% of the total variation in the data. The second principal component explains 19.29% of the total variation. All the first five principal components combined explain 86.422% of the total variation in the data. If the 7,050 observations were projected onto the direction represented by the first principal component, each projected point's distance from their mean value is 1.8573.

Principal components must be uncorrelated with each other for a valid analysis. Using the *ggpairs()* function, the plot created and shown in the PCA section of the appendix shows how there is zero correlation between the principal components. If principal components were correlated, the algorithm would not work since its purpose is to decorrelate them. Decorrelating all explanatory variables in each principal component is important for the future use of external validation in K-means.

Furthermore, the *biplot()* function plots the first two principal components of our data. It represents both the principal component scores and the loading vectors in this display. Although it’s difficult to see the exact observations that represent the scores for the first two principal components, the red arrows indicate the first two principal component loading vectors. For instance, variables such as the number of likes and the number of reactions is located close to one another. Since the two are going in the same direction, there is an indication that they are highly correlated. See the *biplot()* function in the PCA section of the appendix.



Viewing the information from the *pc.out()* function, the first principal component places equal weight on the number of comments, shares, loves, hahas, sads, wows, and angrys. The first component roughly corresponds to the different types of statuses using emojis and post-sharing. It's important to note that the emoji reactions are different from the traditional "likes" that people give for posts. There's much less weight on the number of likes and reactions for the first loading vector. For the second principal component, most of the weight is placed on the number of reactions and the number of likes. The second component corresponds to the overall number of likes and reactions a post was given. These weights correspond to the directions of each predictor in the biplot.

K-Means Clustering

Previously in the exploratory section, some separability between the types of posts was seen. The K-means clustering will dive deeper into the different possible pre-specified clusters and determine what results are best. Library *factoextra* in R-studio allows for the use of the *eclust()* function to perform K-means clustering. The prespecified number of clusters that are used in this section are K=2 through K=7. Using the *nstart* argument in R-studio is required to run the K-means algorithm with multiple initial cluster assignments. Usually using the *nstart* improves the clustering solutions, but it does not for this data since *nstart*=1 and *nstart*=20 interestingly yielded the same total within-cluster sum of squares values. Examining the number of clusters K that are going to be optimal can be determined through the plots for all values of K and validation methods.

```
table(x$status_type) #number of types of posts
##  link  photo status  video
##    63   4288    365   2334
```

Once again, it will be crucial to refer to this table when trying to determine the optimal number of clusters. Shown in the K-means section of the appendix, the plots for all values of K indicate that the most well-clustered plots are for K=2, K=3, and lastly K=4. Clusters for K=2 and K=3 seem to have the most compact clusters, with no overlapping. As seen in the plot of K=4, there is some overlapping between clusters 2 (green) and 3 (blue). This indicates that some of the observations are placed in the wrong cluster. Many observations are also far away from their centroids, more than likely due to the extreme values in this data set. For clusters K=5 through K=7, the observations are poorly clustered even more and continue to overlap worse than the previous values of K.

Minimizing the total within-cluster variation summed across all K clusters must be as small as possible for a great clustering solution. The total sum of squares (Total SS) minus the total within-cluster sum of squares (Total Within SS) results in the within-cluster variation for each K-means cluster. It's important to note that the total sum of squares across all values of K is equal to 63,441. In similar order to K=2 through K=7, the calculated result for within-cluster variation for each value of K is 15,169, 25,073, 30,975, 34,304, 38,077, 39,489. The within-cluster variation for K=2 obtained the smallest result of 15,169, meaning it's the most well-clustered out of all values of K. This corresponds with the previous findings in the plots when viewing the most well-clustered solutions. Calculations for the within-cluster variation can be seen in the K-means appendix.

Performing internal and external validation will assist in figuring out the optimal number of clusters. The largest average silhouette coefficient width (closest value to 1) is the best measure for this process. To begin, the parameter K is not specified when using the *eclust*() function to implement the k-means. The *nboot* parameter corresponds to the number of replicates for the gap statistic replication; *nboot*=40 with 20 different random initializations of cluster assignments selected. After not specifying the k parameter, the *eclust()* function provided an automatic selection of K value K=5 which can also be seen in the gap statistic. This is close to the total number of labels for the data set, which is 4. Clusters one (red), 3 (green), and four (blue) all overlap with each other. This indicates that it is not a great clustering solution and should not be optimal. The gap statistic and the number of clusters are shown in the K-means appendix.

Next, the average silhouette coefficient width was produced for a single clustering solution, specifically for K=4 since this is the number of expected labels that are in the data set. The average silhouette coefficient width for K=4 is 0.7575424 and is the same K=4 plot as earlier. For more internal validation, the specification for the level of K to be 1 through 10 was used. After specification, the gap statistic determined the optimal number of clusters to be K=2. The plot for K=2 is also the same as the one mentioned earlier. Moreover, the average silhouette coefficient width for K=2 is 0.8157665 which can be seen in the cluster silhouette plot. This is quite higher than the width of the single clustering solution of K=4, but K=2 does not correspond with the number of expected labels. Information regarding the silhouette coefficients can be found in the K-means appendix.

Continuing with external validation, the principal component analysis data for the first five principal components will be used to evaluate the optimal number of clusters. Based on the principal component analysis data, the gap statistic determined the optimal number of clusters to be K=4 and the average silhouette coefficient width is 0. 7579541. This result is slightly better than the single clustering solution of K=4 has an average silhouette width of 0.7575424 and corresponds with the expected number of labels. Also, the plot of the K-means cluster for K=4 on the principal component analysis data looks much different compared to the K=4 plot but still obtained some overlapping clusters. The external validation, average silhouette coefficient width, and plot of the K-means cluster can be seen in the K-means appendix.

Hierarchical Clustering

Hierarchical clustering usually provides a tree-based representation of the observations, called a dendrogram. Unfortunately, this data set has 7,050 observations and is too large to properly view the dendrogram. Due to this, the focus of hierarchical clustering will revolve around the viewing of the gap statistic and the silhouette coefficients of the different types of linkages to determine the best clustering solution. This method will possibly give insight into which type of linkage may give the best results depending on the results given.

Starting off, this algorithm requires data to be scaled since it’s sensitive to outliers. The *fviz_nbclust()* function in R-studio shows the gap statistic for each type of linkage which represents the optimal number of K clusters. For complete linkage, the gap statistic determined the optimal number of clusters to be K=3. Both gap statistics for single and average linkage yielded an optimal number of clusters K=2. It’s interesting that none of the previous algorithms have obtained K=3 for the optimal number of clusters whereas average linkage did. View the hierarchical appendix to see the optimal number of clusters.

Like the internal validation process of K-means clustering, using the average silhouette coefficient width will also be used for hierarchical clustering. Average linkage yielded an average silhouette coefficient width of 0.9241791. Considering that the value of this is close to 1, it's relatively well clustered. Complete linkage yielded an average silhouette coefficient width of 0.9200245. Strangely, the computation for the single linkage average silhouette coefficient width was unsuccessful on multiple attempts due to an uncorrectable error. With this being the case, it can be said that average linkage obtained the largest average silhouette coefficient width. The different linkage average silhouette coefficients are shown in the hierarchical appendix.

Interpretations

The principal component analysis gave information on the weights of the loading vectors and the results of the proportion of variation that each of the 9 principal components accounted for. The number of likes and reactions were highly correlated with each other, resulting in both variables being in the same direction on the first and second vectors. Other variables (comments, shares, and types of emojis) were more equally weighted on the first loading vector. Even though 9 principal components were obtained, one of the main difficulties of principal component analysis is determining the true number of principal components that are needed for the data set.

The best average silhouette coefficient for K-means was 0.82 but only for K=2 clusters which also had the smallest within-cluster variation. On the principal component analysis data, it obtained a K=4 number of clusters with a lower average width of 0.75 corresponding with the labels. Determining the best is difficult since there could be an issue with the dominance of photos and videos in the data set. This issue could result in the inconsistency of obtaining the true optimal number of clusters due to many observations for videos and photos versus links and status. The true number of clusters should ideally be K=4.

On the other hand, hierarchical clustering always yields the same results and does not require pre-specification for the number of clusters. Average linkage yielded K=2 number of clusters with an average silhouette coefficient width of 0.9241791. This is slightly larger than the average silhouette coefficient width that complete linkage yielded. In the end, the average widths of hierarchical clustering obtained are much larger than the average widths of the K-means clustering algorithm.

Conclusion

Despite inconsistency in some of the results for clustering, the algorithms for unsupervised learning did form groups. Since the optimal number of clusters seemed to be K=2 based on the results the algorithms gave, it reinforces the assumption that photos and videos are the groups formed in this process. On top of that, the principal component analysis algorithm explained the variation in this data through all nine total principal components. The main issue throughout the algorithmic process was running hierarchical clustering for all linkage methods, especially for single linkage since it did not work. For future analysis, removing the 63 observations for the label 'link' and the 365 observations for 'status' could possibly provide more consistent results for the true optimal number of clusters. Removing some of the 7,050 observations could also lead to easier computations for the hierarchical clustering and representable dendrograms as well.

References

Nassim Dehouche and Apiradee Wongkitrungrueng. Facebook Live as a Direct Selling Channel, 2018, Proceedings of ANZMAC 2018: The 20th Conference of the Australian and New Zealand Marketing Academy. Adelaide (Australia), 3-5 December 2018.

Source: https://archive.ics.uci.edu/ml/datasets/Facebook+Live+Sellers+in+Thailand

Appendix: Source Code

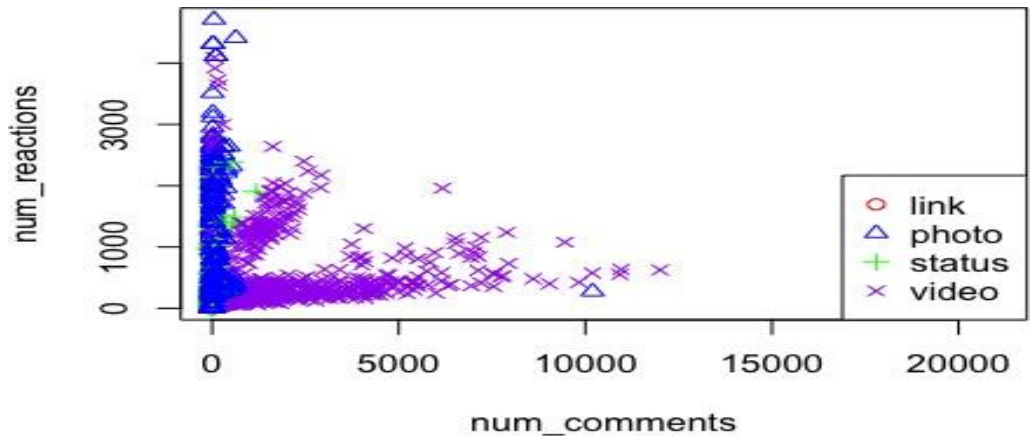
Exploratory Data Analysis (EDA)

```
myfile=read.csv("Live_20210128.csv")
x= data.frame(myfile)
attach(x)

#Removing columns that have no values and others
x$Column1=NULL
x$Column2=NULL
x$Column3=NULL
x$Column4=NULL
x$status_published=NULL
x$status_id=NULL

x$status_type=as.factor(x$status_type)
dim(x)

## [1] 7050   10

table(x$status_type) #description of dimensions and types of posts

##
##   link  photo status  video
##     63   4288    365   2334

cor(x[,2:10]) #correlation matrix

##               num_reactions num_comments num_shares  num_likes num_loves
## num_reactions    1.00000000    0.1508429  0.2507225 0.99492337 0.3050028
## num_comments     0.15084290    1.0000000  0.6406374 0.10168704 0.5212235
## num_shares       0.25072252    0.6406374  1.0000000 0.17249193 0.8200002
## num_likes        0.99492337    0.1016870  0.1724919 1.00000000 0.2093081
## num_loves        0.30500278    0.5212235  0.8200002 0.20930809 1.0000000
## num_wows         0.26775182    0.1623940  0.4076279 0.20779976 0.5087980
## num_hahas        0.17602804    0.3250480  0.3998256 0.12078383 0.5078296
## num_sads         0.07513801    0.2364527  0.1999700 0.05216884 0.2076002
## num_angrys       0.12432606    0.2251844  0.3125126 0.08743065 0.3710013
##                  num_wows num_hahas   num_sads num_angrys
## num_reactions 0.26775182 0.1760280 0.07513801 0.12432606
## num_comments  0.16239399 0.3250480 0.23645268 0.22518437
## num_shares    0.40762786 0.3998256 0.19996997 0.31251256
## num_likes     0.20779976 0.1207838 0.05216884 0.08743065
## num_loves     0.50879804 0.5078296 0.20760019 0.37100127
## num_wows      1.00000000 0.2877562 0.08650272 0.18308651
## num_hahas     0.28775619 1.0000000 0.14142086 0.21191016
## num_sads      0.08650272 0.1414209 1.00000000 0.14207247
## num_angrys    0.18308651 0.2119102 0.14207247 1.00000000

library(corrplot)
```

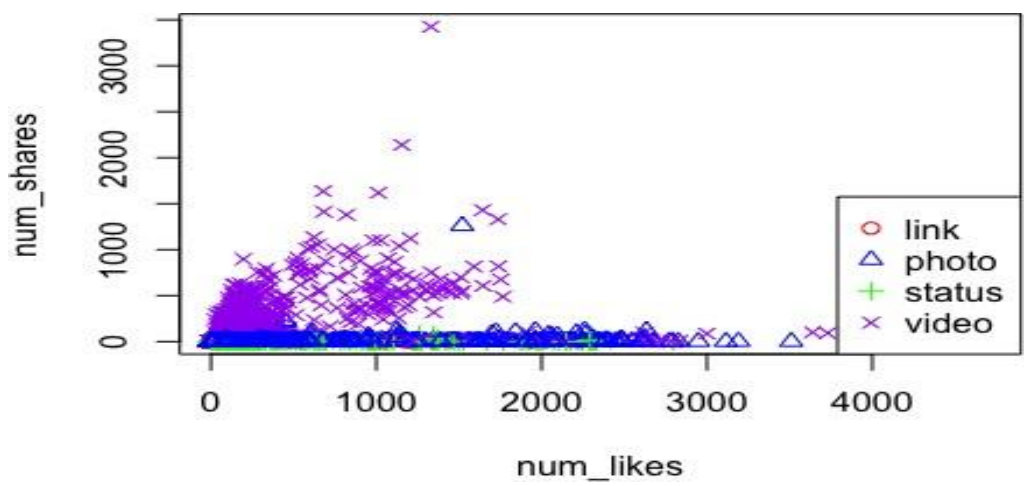
```
## corrplot 0.92 loaded

aplot=cor(x[,2:10])
corrplot(aplot,method="circle",type="upper")

summary(x) #summary of the data

##  status_type   num_reactions     num_comments       num_shares
##  link  :  63   Min.   :   0.0   Min.   :    0.0   Min.   :   0.00
##  photo :4288   1st Qu.:  17.0   1st Qu.:    0.0   1st Qu.:   0.00
##  status: 365   Median :  59.5   Median :    4.0   Median :   0.00
##  video :2334   Mean   : 230.1   Mean   :  224.4   Mean   :  40.02
##                3rd Qu.: 219.0   3rd Qu.:   23.0   3rd Qu.:   4.00
##                Max.   :4710.0   Max.   :20990.0   Max.   :3424.00
##    num_likes         num_loves         num_wows          num_hahas
##  Min.   :   0.0   Min.   :  0.00   Min.   :  0.000   Min.   :  0.0000
##  1st Qu.:  17.0   1st Qu.:  0.00   1st Qu.:  0.000   1st Qu.:  0.0000
##  Median :  58.0   Median :  0.00   Median :  0.000   Median :  0.0000
##  Mean   : 215.0   Mean   : 12.73   Mean   :  1.289   Mean   :  0.6965
##  3rd Qu.: 184.8   3rd Qu.:  3.00   3rd Qu.:  0.000   3rd Qu.:  0.0000
##  Max.   :4710.0   Max.   :657.00   Max.   :278.000   Max.   :157.0000
##     num_sads         num_angrys
## Min.   : 0.0000   Min.   : 0.0000
##  1st Qu.: 0.0000   1st Qu.: 0.0000
##  Median : 0.0000   Median : 0.0000
##  Mean   : 0.2437   Mean   : 0.1132
##  3rd Qu.: 0.0000   3rd Qu.: 0.0000
##  Max.   :51.0000   Max.   :31.0000
```

Plots for Separability

```
plot(num_reactions ~ num_comments, data=x,
     col=c("red","blue","green","purple")[as.integer(x$status_type)],
     pch=c(1,2,3,4)[as.integer(x$status_type)])
legend(x="bottomright",
       legend=c("link","photo","status","video"),
       col=c("red","blue","green","purple"),
       pch=c(1,2,3,4)) #checking separability between reactions and comments
```




```
plot(num_shares~ num_likes, data=x,
     col=c("red","blue","green","purple")[as.integer(x$status_type)],
     pch=c(1,2,3,4)[as.integer(x$status_type)])
legend(x="bottomright",
       legend=c("link","photo","status","video"),
       col=c("red","blue","green","purple"),
       pch=c(1,2,3,4)) #checking separability between shares and likes
```



Principal Component Analysis (PCA)

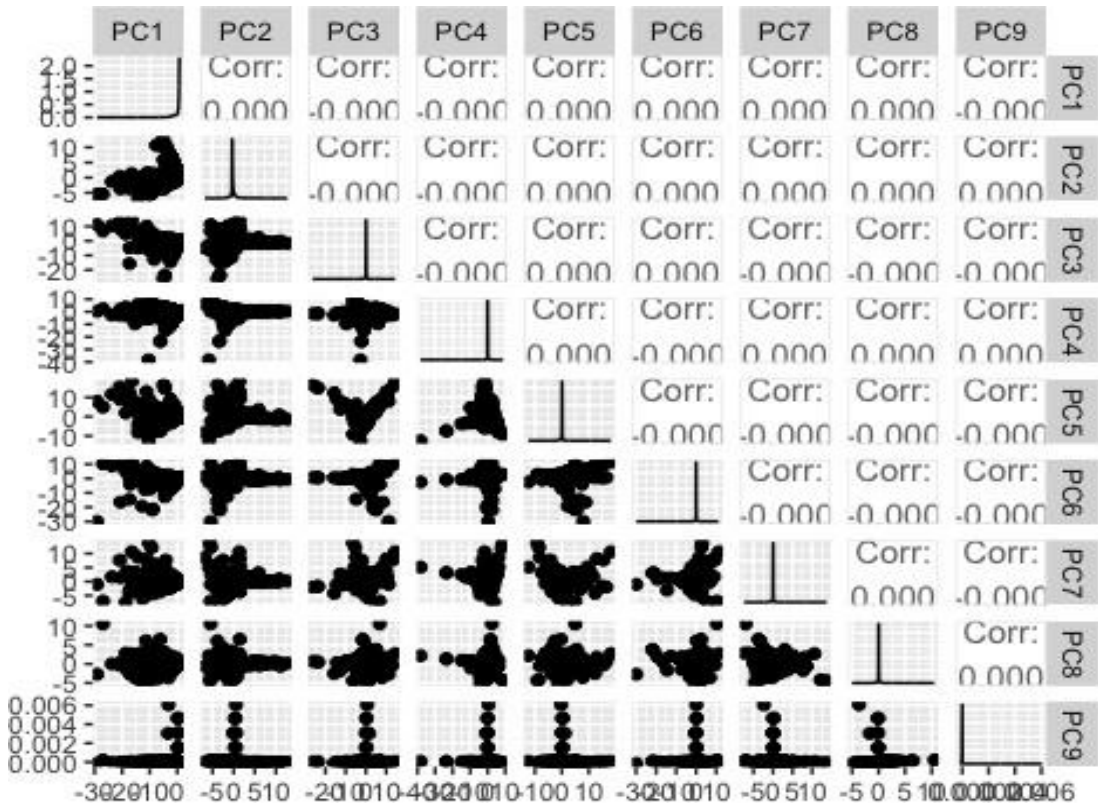
```
library(tidyverse)

library(GGally)

set.seed(1)
pc.out=prcomp(x[,2:10], scale = T) #Using all variables excluding the
status_type
pc.out #information on all 9 principal components and their loading vectors
```

```
## Standard deviations (1, .., p=9):
## [1] 1.8572551618 1.3176044208 0.9823069432  0.9136578979 0.8903080771
## [6] 0.8321728054 0.6167684259 0.3863428675  0.0001283815
##
## Rotation (n x k) = (9 x 9):
##                       PC1          PC2         PC3         PC4         PC5
## num_reactions -0.3090727  0.616916146 -0.07730356  0.01472956 -0.06302969
## num_comments  -0.3494682 -0.242348417 -0.22924824  0.36155545 -0.43032562
## num_shares    -0.4485182 -0.214030388  0.05835103  0.18442535 -0.17310876
## num_likes     -0.2665064  0.652340400 -0.09982847  0.01483545 -0.08079662
## num_loves     -0.4689717 -0.174296364  0.14948666  0.03770100  0.01117834
## num_wows      -0.3136314  0.007425138  0.44066266 -0.11623607  0.62106395
## num_hahas     -0.3201136 -0.150886279  0.16591715  0.09691759  0.17292578
## num_sads      -0.1691169 -0.130696253 -0.82396823 -0.04847701  0.51821957
## num_angrys    -0.2465350 -0.137993385 -0.07153216 -0.89897312 -0.30008690
##                       PC6         PC7         PC8           PC9
## num_reactions -0.02779238 -0.01348180 -0.02091604  7.157732e-01
## num_comments   0.12814505  0.63757433 -0.16661263 -1.057669e-06
## num_shares     0.24351921 -0.39779263  0.68417661  1.759712e-05
## num_likes     -0.03559862  0.02007222  0.03779132 -6.954237e-01
## num_loves      0.08808208 -0.48242169 -0.69388531 -6.186909e-02
## num_wows       0.35799579  0.41920910  0.06392337 -1.348882e-02
## num_hahas     -0.88565019  0.09753375  0.11836068 -6.119324e-03
## num_sads       0.02097750 -0.06033313  0.02016247 -2.469884e-03
## num_angrys    -0.03875319  0.11426052  0.04793893 -1.122400e-03
```

```
summary(pc.out) #this shows the proportion of variance explained
```

```
Importance of components:
                          PC1    PC2    PC3     PC4     PC5     PC6     PC7     PC8       PC9
Standard deviation     1.8573 1.3176 0.9823 0.91366 0.89031 0.83217 0.61677 0.38634 0.0001284
Proportion of Variance 0.3833 0.1929 0.1072 0.09275 0.08807 0.07695 0.04227 0.01658 0.0000000
Cumulative Proportion  0.3833 0.5762 0.6834 0.77613 0.86420 0.94115 0.98342 1.00000 1.0000000
```

```
ggpairs(as.tibble(pc.out$x)) #function for showing that the principal
component are not correlated through ggpairs plot
```

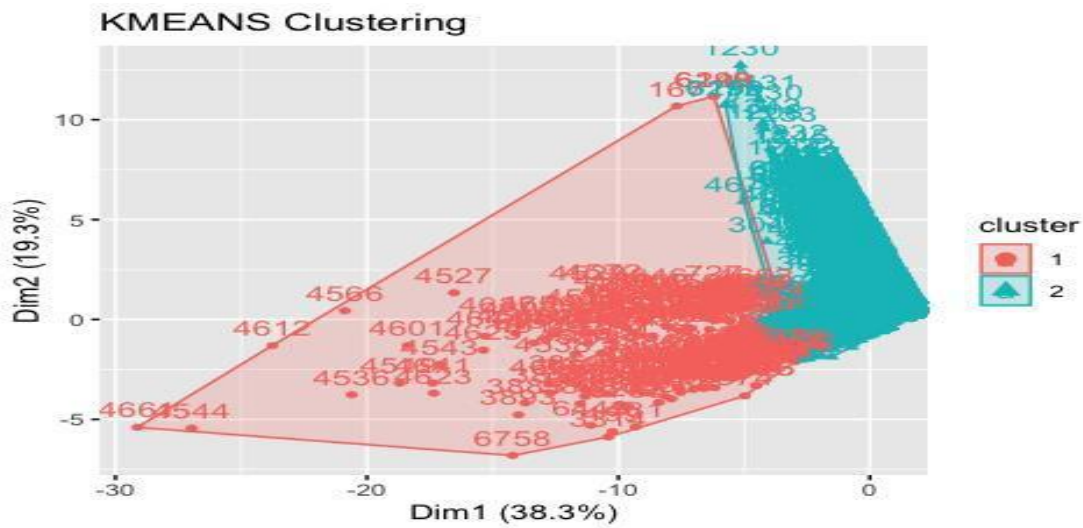



```
biplot(pc.out) #visual representation of loading vectors & their directions
```

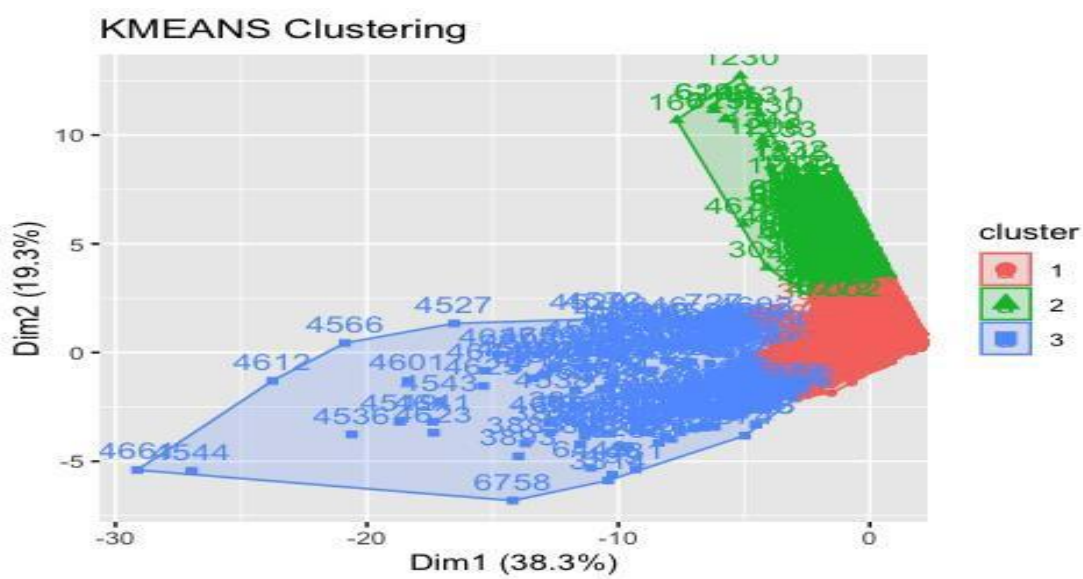
K-means Clustering (K-means)

```
library(factoextra)

fb=scale(x[,2:10]) #scaling the data for numeric variables only

set.seed(1)
km.out.2=eclust(fb,FUNcluster="kmeans",k=2,nstart=50) #pre-specified K=2
```

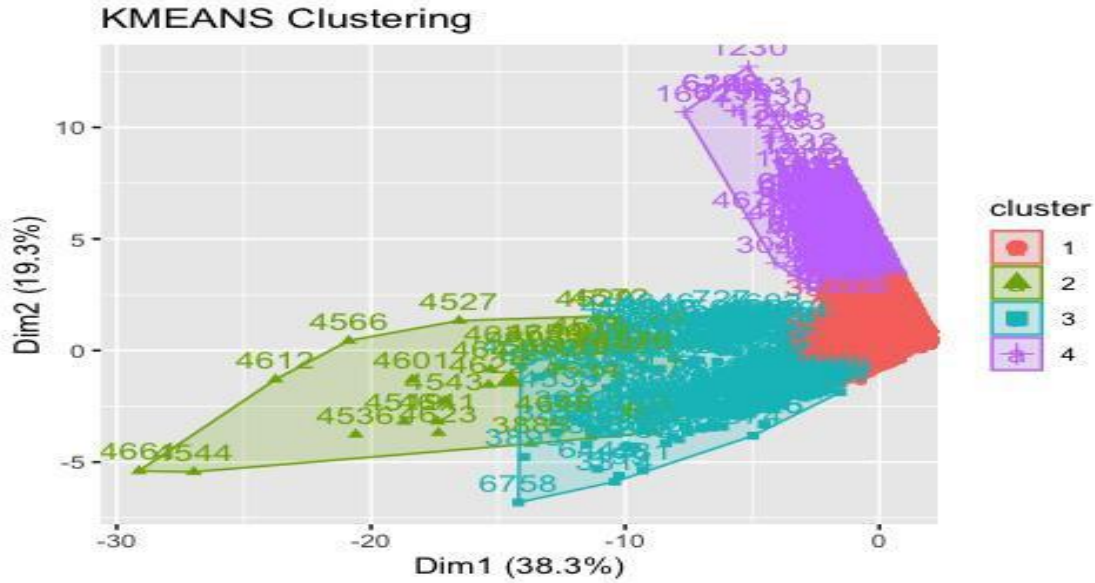
All Plots for K=2-K=7



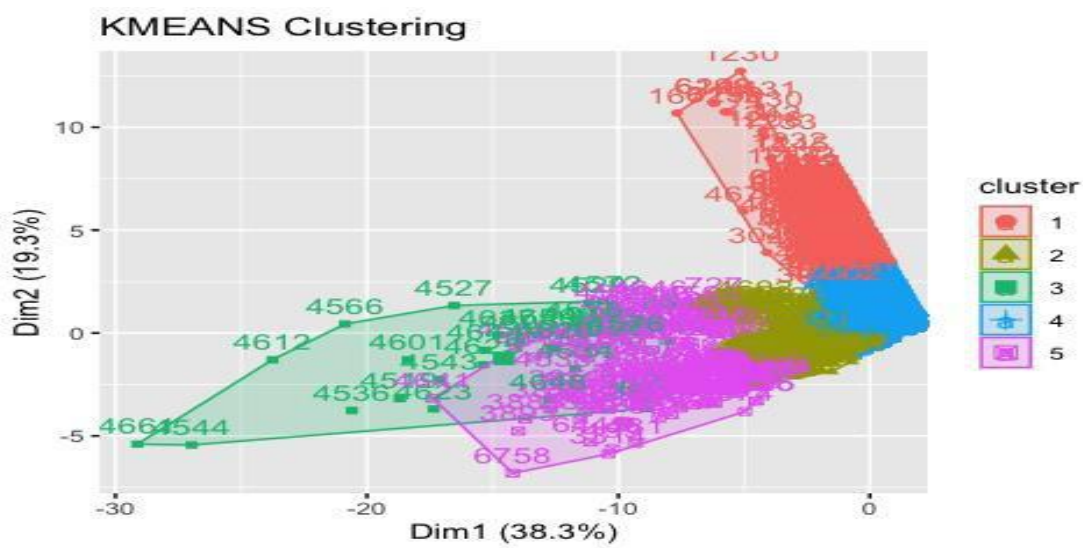

```
set.seed(1)
km.out.3=eclust(fb,FUNcluster="kmeans",k=3,nstart=50) #pre-specified K=3
```



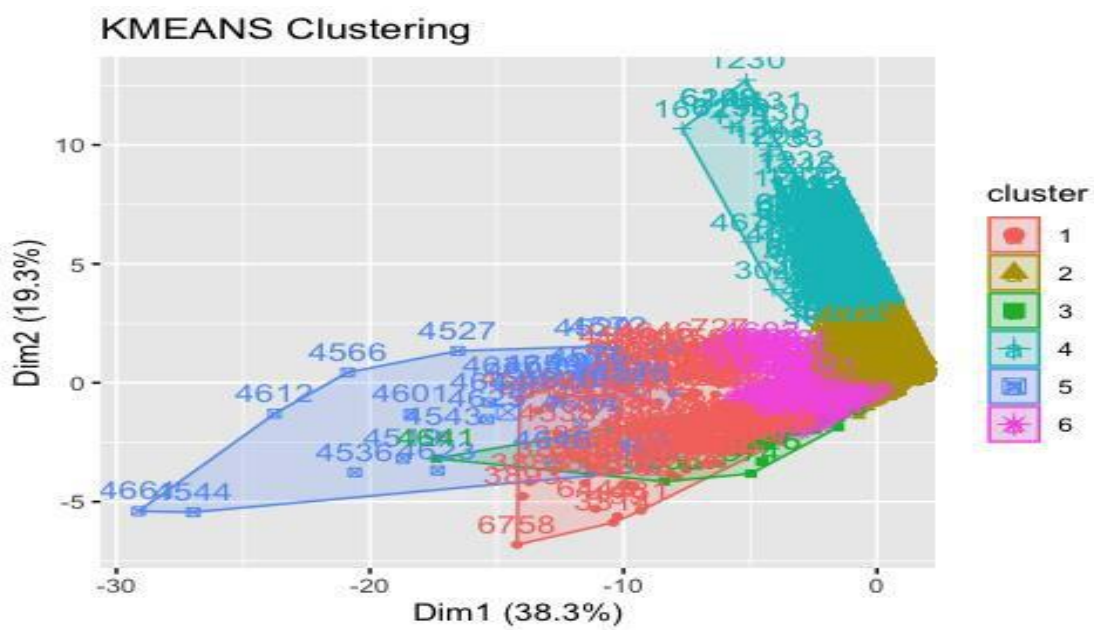

```
library(factoextra)
set.seed(1)
km.out.4=eclust(fb,FUNcluster="kmeans",k=4,nstart=50) #pre-specified K=4
```



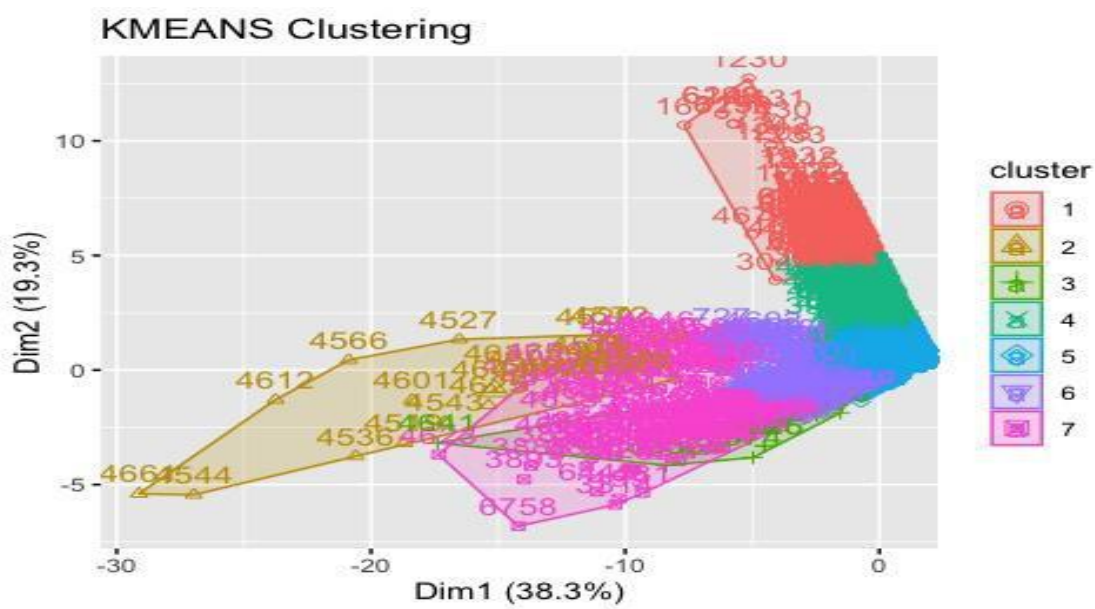

```
set.seed(1)
km.out.5=eclust(fb,FUNcluster="kmeans",k=5,nstart=50) #pre-specified K=5
```




```
set.seed(1)
km.out.6=eclust(fb,FUNcluster="kmeans",k=6,nstart=50) #pre-specified K=6
```




```
set.seed(1)
km.out.7=eclust(fb,FUNcluster="kmeans",k=7,nstart=50) #pre-specified K=7
```



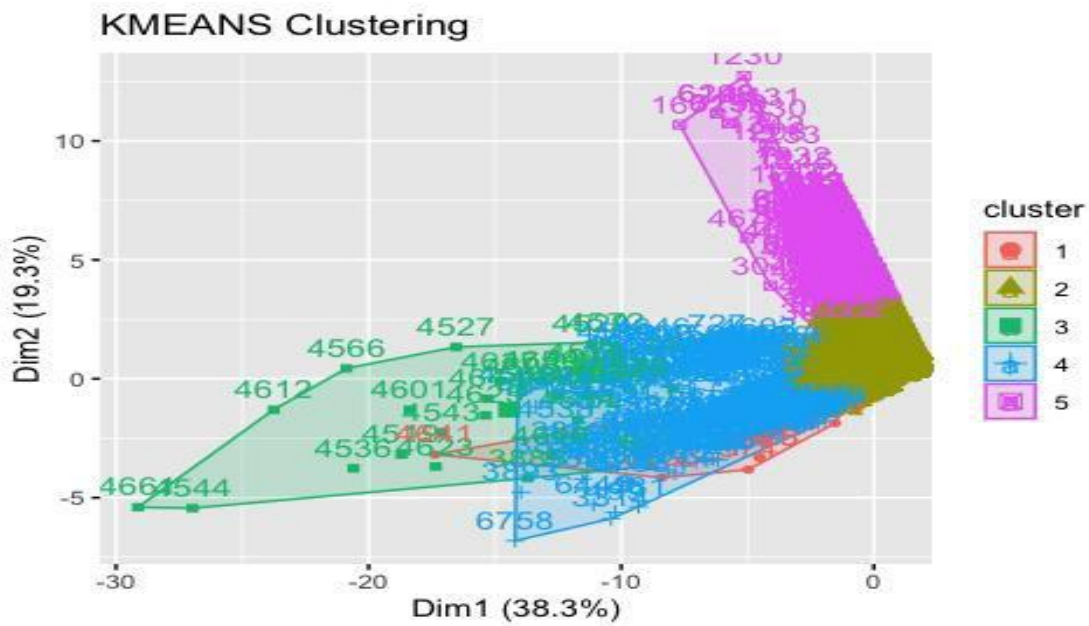
Random Initializations (nstart argument)

```
set.seed(1)
km.out=eclust(fb, FUNcluster = "kmeans", k=4, #random initialization = 1
              nstart=1)

km.out$tot.withinss

## [1] 32465.16 #the total within-cluster sum of squares value for nstart=1
```

```
set.seed(1)
km.out=eclust(fb, FUNcluster = "kmeans", k=4, #random initialization = 20
              nstart=20)

km.out$tot.withinss

## [1] 32465.16 #the total within-cluster sum of squares value for nstart=20
```

Beginning of validation process for K-means algorithm

```
km.out=eclust(fb,FUNcluster="kmeans",nstart=20,nboot=40) #not specifying the
number of K for internal validation
```

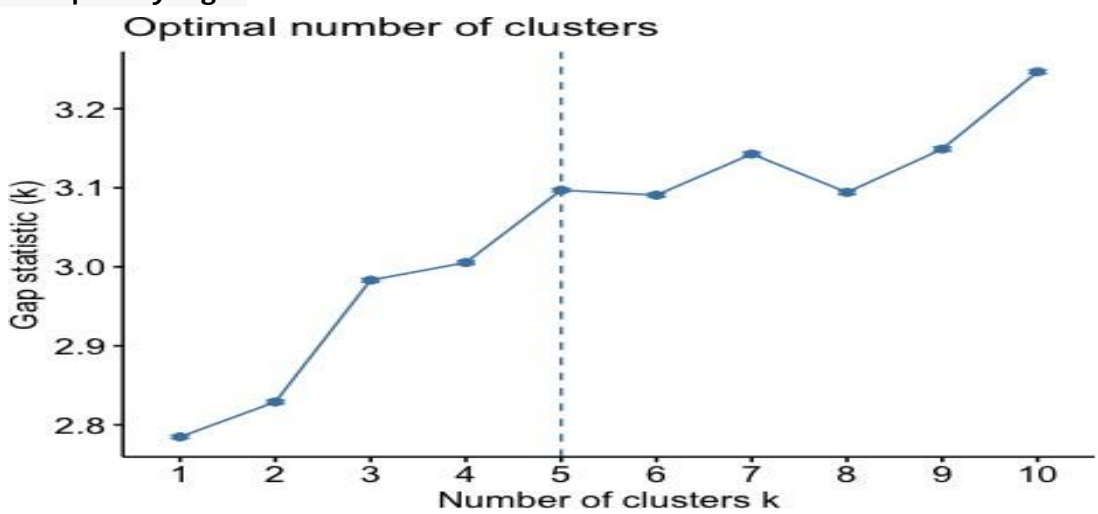



```
km.out

## K-means clustering with 5 clusters of sizes 14, 6246, 33, 366, 391
##
## Cluster means:
##   num_reactions num_comments num_shares   num_likes   num_loves    num_wows
## 1     0.4898070    0.4143758  0.1713441   0.3565127   0.7894608  0.52384912
## 2    -0.2655084   -0.1683183 -0.1780080  -0.2552487  -0.1679978 -0.08865393
## 3     2.7604075    1.6712387  5.4087828   1.9150116   7.0486751 11.74793788
## 4     0.6822261    2.9044507  2.7772473   0.4615504   2.4587208  0.29143822
## 5     3.3522248   -0.1858433 -0.2187288   3.4710215  -0.2410137  0.13312072
##    num_hahas    num_sads num_angrys
## 1  0.3294126 17.28914142  1.8097942
## 2 -0.1031313 -0.08220613 -0.1035305
## 3  7.2596555  1.06170347  2.3041551
## 4  1.2297604  0.78830393  1.6486851
## 5 -0.1281709 -0.13336043 -0.1486993
```

```
## Within cluster sum of squares by cluster:
## [1]  1130.584  6556.407  6359.365 12600.569  1975.863
##  (between_SS / total_SS =  54.9 %)
```

```
library(factoextra)
fviz_nbclust(fb, kmeans,     #gap statistic for not specifying value of K
             nstart = 20,
             nboot = 40,
             method = "gap_stat")
```

Not Specifying K

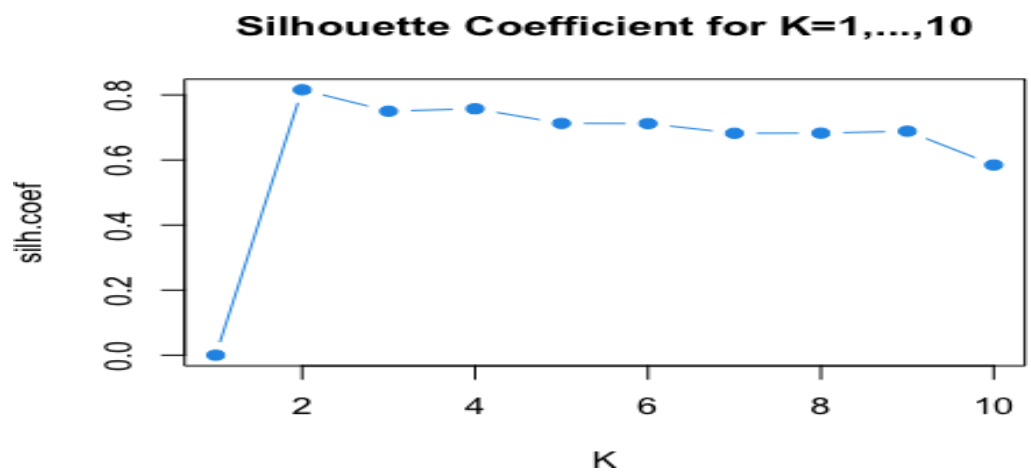



```
set.seed(2) #validation for a single clustering solution
k=4
ec.obj <- eclust(fb,
                 FUNcluster="kmeans",
                 k = k,
                 nstart = 50)

ec.obj$silinfo #silhouette coefficient average width for K=4
```

```
$widths

$clus.avg.widths
[1]  0.82534999  0.05600959 -0.05461577  0.50712653

$avg.width
[1] 0.7575424
```

Internal Validation for K.max=10

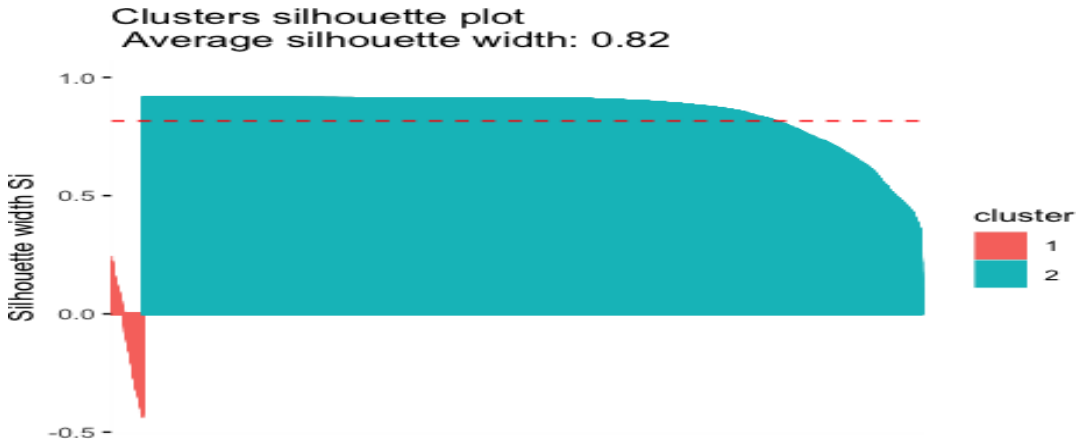
```
set.seed(1)
k.max <- 10
silh.coef <- numeric(k.max)
for (k in 2:10){
  print(k)                        #this is internal validation K-values 1-10
  silh.coef[k] <- eclust(fb,
                         FUNcluster="kmeans",
                         k = k,
                         graph=0,
                         nstart = 50)
}

plot(silh.coef,
     type="b", pch=19, col=4,
     main="Silhouette Coefficient for K=1,...,10",
     xlab="K")
```




```
k=2
ec.obj <- eclust(fb,
                 FUNcluster="kmeans", #average silhouette coefficient from
internal validation
                 k = k,
                 nstart = 50)

ec.obj$silinfo #getting average width for K=2 after k.max=10
```

```
$widths

$clus.avg.widths
[1] -0.09652606  0.85265672

$avg.width
[1] 0.8157665
```

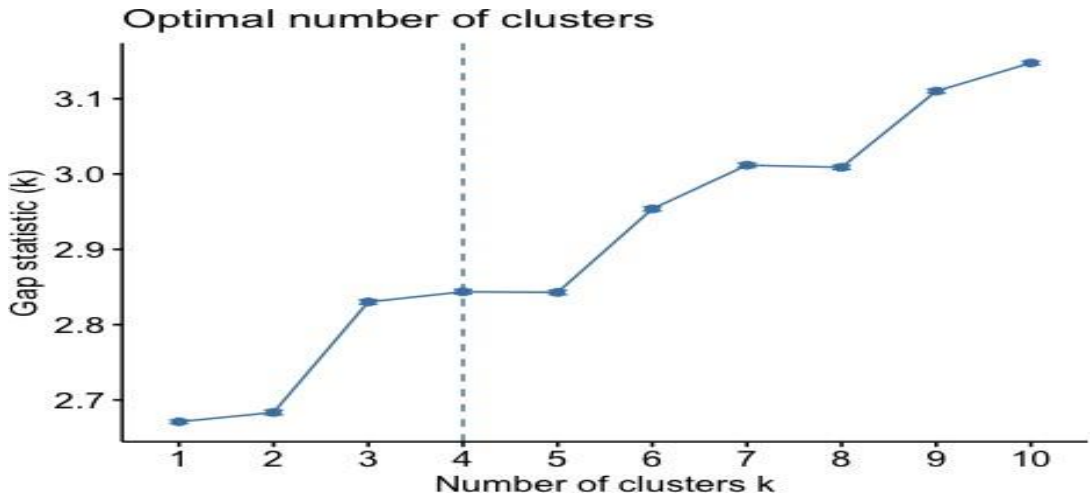
```
library(cluster)
set.seed(1)
sil <- silhouette(ec.obj$cluster, #plot of K=2 internal validation
                  dist(fb))
fviz_silhouette(sil)

##   cluster size ave.sil.width
## 1       1  274         -0.10
## 2       2 6776          0.85
```



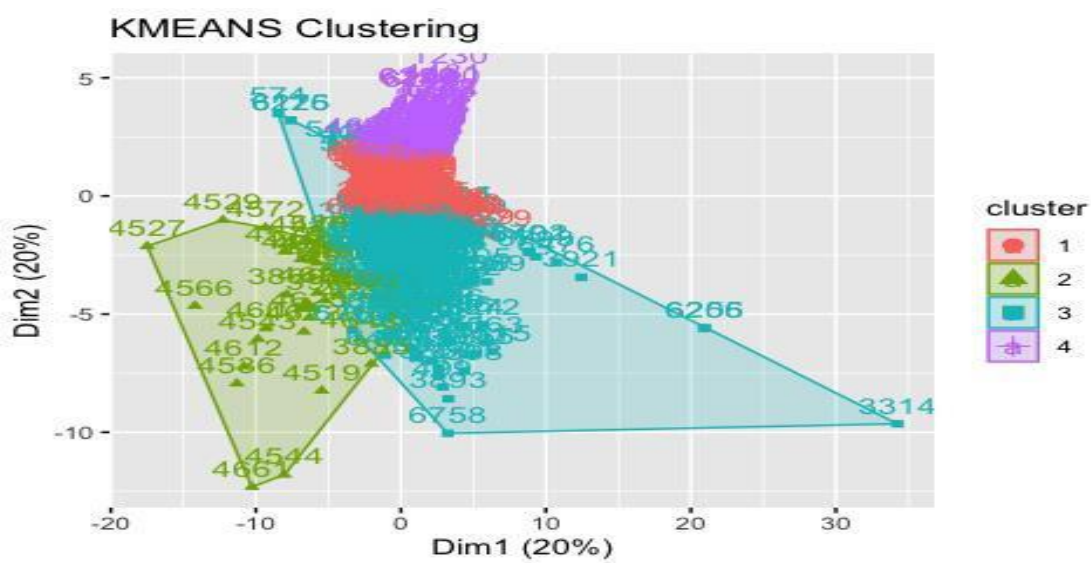
External Validation for K-means on PCA Data

```
pca.obj=prcomp(fb)
dim(pca.obj$x)

## [1] 7050    9 #dimensions of PCA

n.PCs=5 #using first five principal components
pca.data <- pca.obj$x[,c(1:n.PCs)]
```

```
set.seed(1)
fviz_nbclust(pca.data, kmeans, k.max=10,
             nstart = 20,
             method = "gap_stat",
             nboot = 40) #gap statistic for PCA data
```




```
set.seed(1)
km.obj <- eclust(pca.data,
                 FUNcluster = "kmeans",
                 k=4,
                 nstart=50) #plot of PCA data for K-means
```



The average silhouette coefficient width on the PCA data for K-means

```{r,results='show'}
mean(sil[,3])
```

```
[1] 0.7579541
```

Within-Cluster Variation Calculations

Total SS for all K

```{r,results='show'}
km.out$totss #Total sum of squares (Total SS)
```

```
[1] 63441
```

Total Within SS for K=2

```{r,results='show'}
km.out.2$tot.withinss #Total within-cluster sume of squares (Total Within_SS )
```

```
[1] 48271.25
```

Total Within SS for K=3

```{r,results='show'}
km.out.3$tot.withinss #Total within-cluster sume of squares (Total Within_SS )
```

```
[1] 38367.55
```

Total Within SS for K=4

```{r,results='show'}
km.out.4$tot.withinss #Total within-cluster sum of squares (Total Within_SS )
```

```
[1] 32465.16
```

Total Within SS for K=5

```{r,results='show'}
km.out.5$tot.withinss #Total within-cluster sume of squares (Total Within_SS )
```

```
[1] 29136.36
```

Total Within SS for K=6

```{r,results='show'}
km.out.6$tot.withinss #Total within-cluster sume of squares (Total Within_SS )
```

```
[1] 25363.69
```

Total Within SS for K=7

```{r,results='show'}
km.out.7$tot.withinss #Total within-cluster sume of squares (Total Within_SS )
```

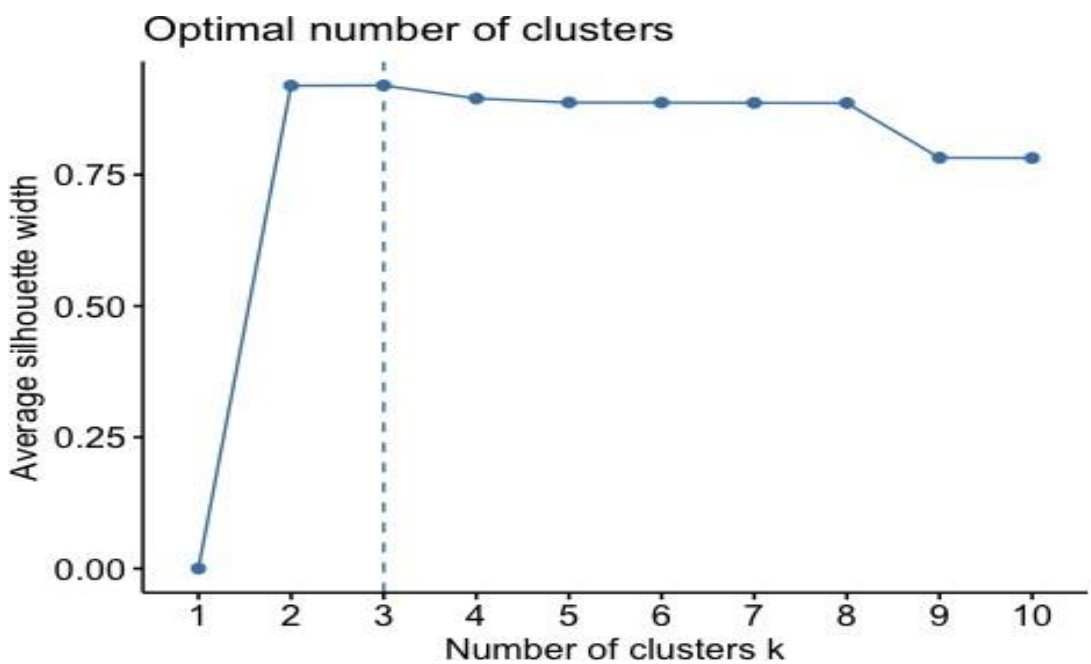
```
[1] 23951.46
```

Hierarchical Clustering (Hierarchical)

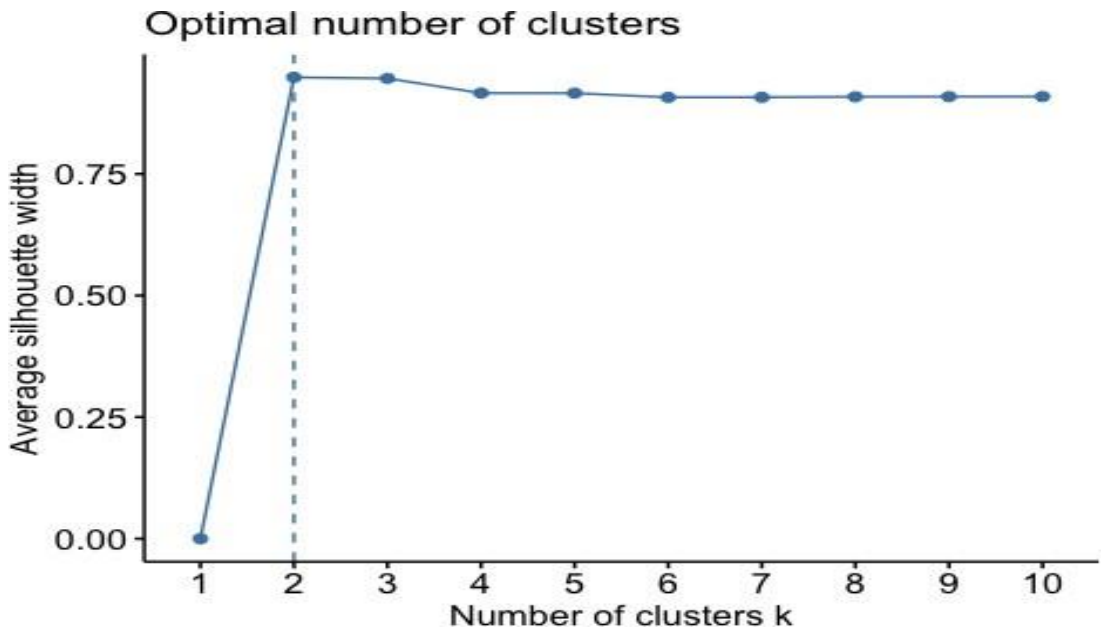
```
fb=scale(x[,2:10]) #scaling the data

fb.labs=x$status_type
fb.labs[1:4]

## [1] video photo video photo
## Levels: link photo status video

table(fb.labs) #labels for posts

## fb.labs
##   link  photo status   video
##     63   4288    365    2334
```

```
set.seed(1)
data.dist=dist(fb) #setting the distance
```

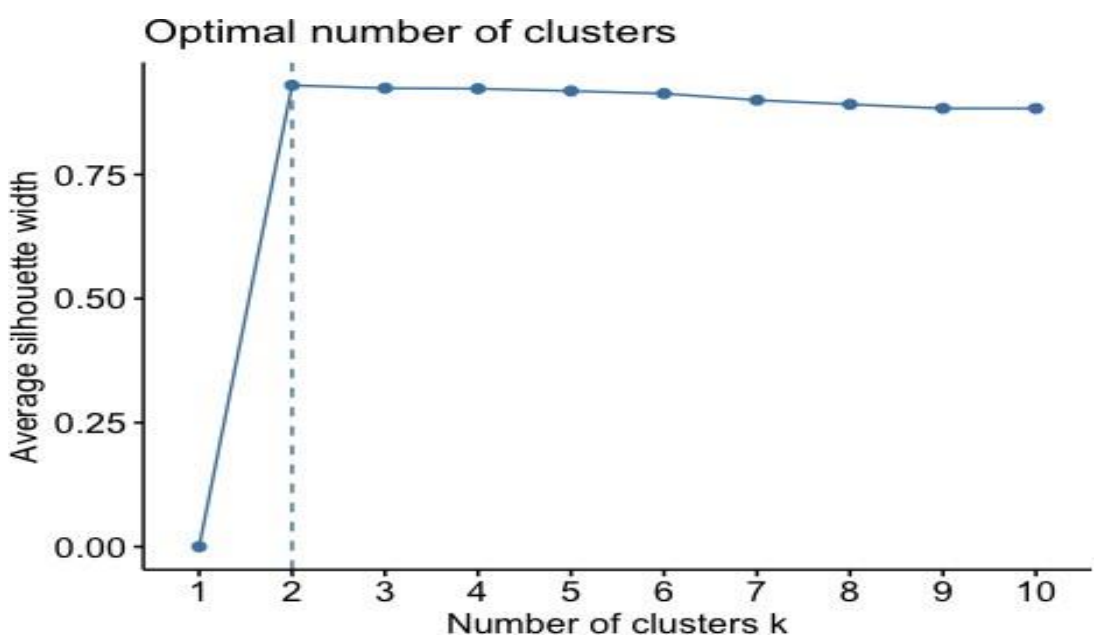
```
library(factoextra)

set.seed(1)
fviz_nbclust(fb, hcut,
             hc_method = "complete") #gap statistic for complete linkage
```



```
set.seed(1)
fviz_nbclust(fb, hcut,
             hc_method = "single") #gap statistic for single linkage
```




```
set.seed(1)
fviz_nbclust(fb, hcut,
             hc_method = "average") #gap statistic for average linkage
```



Average Silhouette Coefficient Widths

Average Linkage

```{r,results='show'}
hc.out=eclust(fb,
              FUNcluster = "hclust",
              hc_method="average")
print(hc.out$silinfo$avg.width)
```

```
Clustering k = 1,2,..., K.max (= 10): .. done
Bootstrapping, b = 1,2,..., B (= 100)  [one "." per sample]:
.................................................. 50
.................................................. 100
[1] 0.9241791
```

Complete Linkage

```{r,results='show'}
hc.out=eclust(fb,
              FUNcluster = "hclust",
              hc_method="complete")
print(hc.out$silinfo$avg.width)
```

```
Clustering k = 1,2,..., K.max (= 10): .. done
Bootstrapping, b = 1,2,..., B (= 100)  [one "." per sample]:
.................................................. 50
.................................................. 100
Warning: The `<scale>` argument of `guides()` cannot be `FALSE`. Use "none" instead as of
ggplot2 3.3.4.[1] 0.9200245
```

Single Linkage (Error)

```{r,results='show'}
hc.out=eclust(fb,
              FUNcluster = "hclust",
              hc_method="single") #error for single linkage
print(hc.out$silinfo$avg.width)
```

```
Clustering k = 1,2,..., K.max (= 10): .. done
Bootstrapping, b = 1,2,..., B (= 100)  [one "." per sample]:
.................................................. 50
.................................................. 100
Error: C stack usage  7969248 is too close to the limit
NULL
```